COURSE 7: DATA ANALYSIS WITH PYTHON

MODULE 1: IMPORTING DATASETS – DATA ANALYSIS WITH PYTHON

1.1 COURSE INTRODUCTION

Python is one of the most widely used programming languages for Data Science because of its simplicity, flexibility, and large ecosystem of libraries.

In this course, you learn how to analyze data using Python and important libraries such as:

- NumPy
- Pandas
- Scikit-learn

These libraries are heavily used in industries like:

- Healthcare
- Finance
- Insurance
- Internet companies
- Research and analytics

1. What You Learn in This Course

Module 1 – Importing Datasets

- Understanding datasets
- Importing datasets into Python
- Using Pandas for analysis
- Understanding data types
- Getting dataset summaries

Module 2 – Data Wrangling

- Handling missing values
- Data formatting
- Data normalization
- Data preprocessing

Module 3 – Exploratory Data Analysis (EDA)

- Descriptive statistics
- GroupBy operations
- Correlation analysis
- Statistical exploration

Module 4 – Model Development

- Linear regression
- Polynomial regression
- Pipelines
- Model evaluation

Module 5 – Model Evaluation & Refinement

- Overfitting and underfitting
- Ridge regression
- Grid search
- Model selection

Final Project

- Real-world data analysis project
- Building predictive models

1.2 UNDERSTANDING THE DATASET

Used Car Price Dataset

The course uses a dataset containing information about used cars.

Dataset Format

- Stored as a CSV file
- CSV = Comma Separated Values
- Each row represents one record
- Columns represent attributes/features

1. Important Dataset Features

Some important columns include:

Attribute	**Meaning**
symboling	Insurance risk level
normalized-losses	Average loss payment per insured vehicle
make	Car manufacturer
price	Car price (target variable)

2. Target Variable

The **Price** column is the:

- Target value
- Label
- Variable we want to predict

All other columns are:

- Features
- Predictors

Goal:
Predict car price using other car characteristics.

1.3 PYTHON PACKAGES FOR DATA SCIENCE

Python libraries simplify data analysis by providing ready-made functions and tools.

Libraries are divided into 3 major categories:

1. Scientific Computing Libraries

Pandas

Used for:

- Data manipulation
- Data cleaning
- Data analysis

Main structure:

- DataFrame (table with rows and columns)

Features:

- Fast access to structured data
- Easy indexing
- Data filtering and grouping

NumPy

Used for:

- Numerical operations
- Arrays
- Matrix calculations
- Fast mathematical processing

Key Feature:
Efficient array computation.

SciPy

Used for:

- Advanced mathematics
- Scientific calculations
- Optimization
- Statistics

2. Data Visualization Libraries

Matplotlib

Used for:

- Graphs
- Charts
- Plots

Features:

- Highly customizable
- Most widely used plotting library

Seaborn

Built on top of Matplotlib.

Used for:

- Heatmaps
- Time series plots
- Violin plots
- Statistical graphics

3. Machine Learning Libraries

Scikit-learn

Used for:

- Regression
- Classification
- Clustering
- Predictive modeling

Built on:

- NumPy
- SciPy
- Matplotlib

Statsmodels

Used for:

- Statistical testing
- Data exploration
- Statistical modeling

1.4 IMPORTING AND EXPORTING DATA IN PYTHON

Data Acquisition

Data acquisition means:

- Loading data into Python
- Reading data from files or databases

Two important things when importing data:

Factor	Meaning
Format	How data is encoded
File Path	Where data is stored

1. Common File Formats

Format	Description
CSV	Comma-separated values
JSON	JavaScript Object Notation
XLSX	Excel file
HDF	Hierarchical Data Format

2. Reading CSV Files Using Pandas

Import Pandas

```
import pandas as pd
```

Load CSV File

```
df = pd.read_csv("file.csv")
```

Load CSV Without Headers

```
df = pd.read_csv("file.csv", header=None)
```

3. Viewing Dataset

View First Rows

```
df.head()
```

Default:
Displays first 5 rows.

View Last Rows

```
df.tail()
```

Displays last 5 rows.

4. Assign Column Names

```
headers = ["symboling", "normalized-losses", "make"]
df.columns = headers
```

Purpose:
Replace default integer column names with meaningful names.

5. Exporting Data

Save DataFrame to CSV

```
df.to_csv("automobile.csv")
```

Purpose:
Save processed dataset.

1.5 GETTING STARTED WITH DATA ANALYSIS

After importing data, the next step is exploring it.

1. Data Types in Pandas

Common data types:

Pandas Type	**Meaning**
object	Text/String
int	Integer
float	Decimal
datetime	Date & Time

Checking Data Types

```
df.dtypes
```

Purpose:

- Identify incorrect data types
- Understand what operations can be applied

Example:
If price is stored as object instead of float, mathematical operations may fail.

2. Statistical Summary

Using describe()

```
df.describe()
```

Provides:

- Count
- Mean
- Standard deviation
- Minimum
- Maximum
- Quartiles

Include All Columns

df.describe(include="all")

Also summarizes object-type columns.

Additional metrics:

- Unique values
- Most frequent value
- Frequency count

3. Dataset Summary

Using info()

```
df.info()
```

Provides:

- Column names
- Data types
- Non-null counts
- Memory usage

4. Replacing Missing Values

```
df = df.replace("?", np.nan)
```

Purpose:
Convert missing placeholders into proper NaN values.

Missing Values

NaN means:

- Not a Number
- Missing or unavailable data

1.6 ACCESSING DATABASES WITH PYTHON

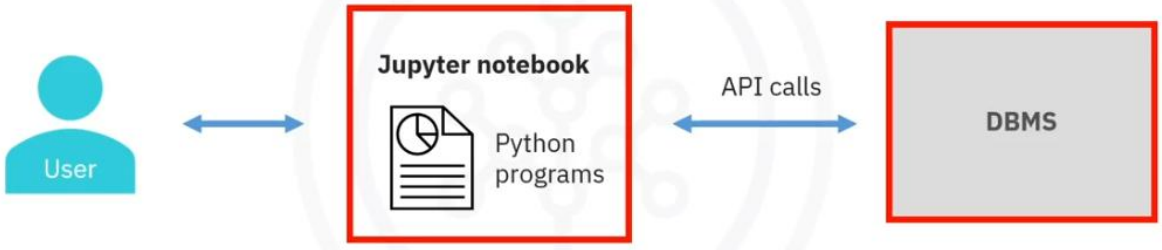
Python can connect to relational databases using APIs.

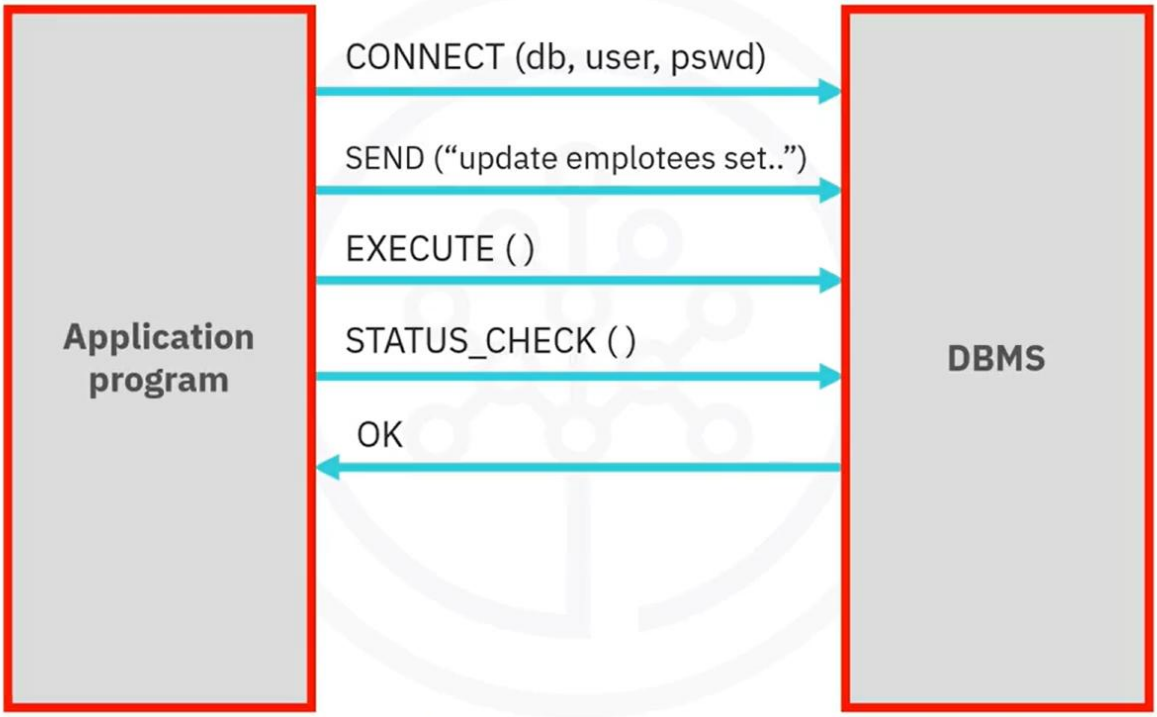
1. SQL APIs

An API (Application Programming Interface) allows programs to communicate with databases.

Operations include:

- Connecting to DBMS
- Sending SQL queries
- Receiving results
- Handling errors





2. Python DB-API

DB-API is Python’s standard interface for relational databases.

Main components:

Component	**Purpose**
Connection Object	Connect to database
Cursor Object	Execute queries

Connection Object Methods

Method	Purpose
cursor()	Create cursor
commit()	Save transaction
rollback()	Undo transaction
close()	Close connection

Example Workflow

Step 1: Import Database Module

```
import sqlite3
```

Step 2: Connect to Database

```
conn = sqlite3.connect("database.db")
```

Step 3: Create Cursor

```
cursor = conn.cursor()
```

Step 4: Execute Query

```
cursor.execute("SELECT * FROM table")
```

Step 5: Fetch Results

```
results = cursor.fetchall()
```

Step 6: Close Connection

```
conn.close()
```

Important:
Always close database connections to free resources.

1.7 LAPTOP PRICING DATASET OVERVIEW

Another dataset used in labs is the Laptop Pricing dataset.

1. **Dataset Parameters**

Feature	**Description**
Manufacturer	Laptop company
Category	Gaming, Notebook, etc.
GPU	GPU manufacturer
OS	Operating system
CPU_core	Processor type
Screen_Size_cm	Screen size
CPU_frequency	CPU speed
RAM_GB	RAM size
Storage_GB_SSD	SSD storage
Weight_kg	Weight
Price	Laptop price

2. Encoded Numerical Values

Category Encoding

Category	**Value**
Gaming	1
Netbook	2
Notebook	3
Ultrabook	4

Category	Value
Workstation	5

GPU Encoding

GPU	Value
AMD	1
Intel	2
NVidia	3

OS Encoding

OS	Value
Windows	1
Linux	2

CPU Encoding

CPU	Value
Intel i3	3
Intel i5	5
Intel i7	7

1.8 IMPORTANT PANDAS COMMANDS CHEAT SHEET

Task	Code
Read CSV	pd.read_csv()
View first rows	df.head()
View last rows	df.tail()
Assign headers	df.columns = headers
Replace missing values	df.replace()
Check data types	df.dtypes
Statistical summary	df.describe()
Dataset info	df.info()
Save CSV	df.to_csv()

LAB: IMPORTING DATA SETS - USED CARS PRICING

LAB: IMPORTING DATASETS - LAPTOP PRICING

MODULE 2: DATA WRANGLING

2.1 INTRODUCTION TO DATA PRE-PROCESSING

1. What is Data Pre-processing?

Data pre-processing is the process of converting raw data into a clean, organized, and meaningful format so it can be analyzed effectively.

It is also called:

- Data Cleaning
- Data Wrangling
- Data Transformation

Raw data often contains:

- Missing values
- Incorrect formats
- Different units
- Duplicate entries
- Inconsistent naming conventions

Before performing analysis or building machine learning models, the data must be cleaned and standardized.

2. Main Topics Covered in Data Wrangling

The module focuses on:

1. Handling Missing Values
2. Data Formatting
3. Data Normalization
4. Binning
5. Converting Categorical Variables into Numerical Variables

3. Working with Columns in Pandas

In Python, data manipulation is usually done column-wise.

Each column in a Pandas DataFrame is called a **Series**.

Example:

```
df['symboling']
df['body-style']
```

You can also perform operations on entire columns.

Example:

```
df['symboling'] = df['symboling'] + 1
```

This adds 1 to every value in the column.

2.2. HANDLING MISSING VALUES

1. What are Missing Values?

A missing value occurs when no data is stored for a feature.

Common representations:

- NaN
- ?
- N/A
- Blank cells
- 0 (sometimes)

Example:

NaN

2. Strategies for Handling Missing Data

Option 1: Retrieve the Original Data

Ask the data provider to supply the correct values.

Option 2: Remove Missing Data

A. Remove Entire Row

Useful when only a few rows contain missing values.

B. Remove Entire Column

Useful when a column contains too many missing values.

Option 3: Replace Missing Data

Replace with Mean

Used for numerical variables.

Example:

```
mean = df['normalized-losses'].astype('float').mean()
```

Replace with Mode

Used for categorical variables.

Example:

```
MostFrequentEntry = df['fuel-type'].value_counts().idxmax()
```

Option 4: Leave Data as Missing

Sometimes missing values themselves are meaningful.

3. Dropping Missing Values in Pandas

Syntax

```
df.dropna()
```

Drop Rows with Missing Values

```
df.dropna(axis=0)
```

Drop Columns with Missing Values

```
df.dropna(axis=1)
```

Modify DataFrame Directly

```
df.dropna(inplace=True)
```

Example:

```
df.dropna(subset = [“price”], axis = 0, inplace = True)
```

4. Replacing Missing Values

Replace NaN with Mean

```
mean = df[“Normalized-losses”]. mean()

df[“Normalized-losses”]. replace (np.nan, mean)
```

Replace NaN with Mode

```
MostFrequentEntry = df['fuel-type'].value_counts().idxmax()

df['fuel-type'].replace(np.nan, MostFrequentEntry, inplace=True)
```

2.3. DATA FORMATTING

What is Data Formatting?

Data formatting means converting data into a common standard format.

Examples:

- NY
- N.Y
- New York

These should often be standardized into one format.

1. **Unit Conversion Example**

Convert Miles per Gallon to Liters per 100 km

Formula:

L/100km = 235 / mpg

Implementation:

```
df['city-L/100km'] = 235 / df['city-mpg']
```

2. Renaming Columns

Syntax

```
df.rename(columns={'old_name':'new_name'}, inplace=True)
```

Example:

```
df.rename(columns={'city-mpg':'city-L/100km'}, inplace=True)
```

3. Understanding Data Types

Common Pandas Data Types:

Data Type	Meaning
object	Strings/Text
int64	Integer values
float64	Decimal values
datetime64	Date & Time

4. Checking Data Types

Syntax

```
df.dtypes
```

Purpose:

- Detect incorrect data types
- Ensure columns are suitable for analysis

5. Fixing Incorrect Data Types

Syntax

```
df[['price']] = df[['price']].astype('float')
```

Example:

```
df['price'] = df['price'].astype('int')
```

2.4 DATA NORMALIZATION

1. What is Normalization?

Normalization scales numerical data into similar ranges.

Purpose:

- Fair comparison between variables
- Better machine learning performance
- Removes bias caused by large ranges

2. Why Normalization is Important

Example:

- Age range: 0–100
- Income range: 0–500000

Without normalization:

- Income dominates analysis due to larger values.

After normalization:

- Both variables contribute fairly.

3. Types of Normalization

A. Simple Feature Scaling

Formula

$$x_{new} = \frac{x_{old}}{x_{max}}$$

Python

```
df['length'] = df['length'] / df['length'].max()
```

B. Min-Max Scaling

Formula

$$x_{new} = \frac{x - x_{min}}{x_{max} - x_{min}}$$

Python

```
df['length'] = (df['length'] - df['length'].min()) / \
               (df['length'].max() - df['length'].min())
```

C. Z-Score Normalization

Formula

$$x_{new} = \frac{x - \mu}{\sigma}$$

Where:

- μ= Mean
- σ= Standard Deviation

Python

```
df['length'] = (df['length'] - df['length'].mean()) / \
        df['length'].std()
```

2.5 BINNING

1. What is Binning?

Binning groups continuous numerical values into categories.

Example:

- Low Price
- Medium Price
- High Price

2. Advantages of Binning

- Simplifies analysis
- Improves model accuracy
- Better visualization
- Easier interpretation

3. Creating Bins in Python

Step 1: Create Bin Ranges

```
bins = np.linspace(min(df['price']),
                   max(df['price']),
                   4)
```

Step 2: Define Labels

```
group_names = ['Low', 'Medium', 'High']
```

Step 3: Create Binned Column

```
df['price-binned'] = pd.cut(df['price'],
                            bins,
                            labels=group_names,
                            include_lowest=True)
```

4. Histograms

Histograms are used to visualize:

- Data distribution
- Frequency of values
- Bin comparison

Observation from used-car dataset:

- Most cars fall into low-price category.
- Very few cars are high-price vehicles.

2.6 CATEGORICAL VARIABLES

1. What are Categorical Variables?

Variables containing labels or categories instead of numbers.

Examples:

- Fuel Type (Gas, Diesel)
- Color

- Brand
- Transmission Type

2. Why Convert Categories to Numbers?

Machine learning algorithms usually require numerical input.

Text values cannot be directly processed by most models.

3. One-Hot Encoding

Concept

Create separate columns for each category.

Example:

Fuel Type	Gas	Diesel
Gas	1	0
Diesel	0	1

4. Creating Dummy Variables in Pandas

Syntax

```
pd.get_dummies()
```

Example:

```
dummy_variable = pd.get_dummies(df['fuel-type'])
```

Combine Dummy Variables with Original DataFrame

```
df = pd.concat([df, dummy_variable], axis=1)
```

2.7 LAB OBJECTIVES

1. Used Cars Pricing Lab

You learned to:

- Handle missing values
- Correct data formatting
- Normalize data
- Prepare datasets for analysis

LAB: DATA WRANGLING - USED CARS PRICING

2. Laptop Pricing Lab

You practiced:

- Data loading
- Data cleaning
- Data formatting
- Data normalization
- Exploratory preprocessing

LAB: DATA WRANGLING - LAPTOP PRICING

FINAL SUMMARY

Data wrangling is one of the most important steps in the data analysis process. Clean, standardized, and properly formatted data improves:

- Accuracy
- Visualization
- Statistical analysis
- Machine learning model performance

This module builds the foundation for:

- Exploratory Data Analysis (EDA)
- Machine Learning
- Predictive Modeling
- Data Visualization
- Data Pipelines

MODULE 3: EXPLORATORY DATA ANALYSIS (EDA)

WHAT IS EDA?

Exploratory Data Analysis (EDA) is used to:

- Understand the dataset
- Find patterns and relationships
- Detect outliers and missing values
- Identify important variables affecting the target variable

Example:
In the car pricing dataset, the main goal is:

Which car features most affect car price?

3.1 DESCRIPTIVE STATISTICS

1. describe() Function

Used for numerical summaries.

```
df.describe()
```

Gives:

- Mean
- Count
- Standard deviation
- Min/Max

- Quartiles (25%, 50%, 75%)

2. value_counts() Function

Used for categorical variables.

```
drive_wheels_cnts = df["drive-wheels"].value_counts()
drive_wheels_cnts.rename(columns = {“drive-wheels” : “value_counts”}, inplace = True)
drive_wheels_cnts.index.name = “drive-wheels”
```

Example Output:

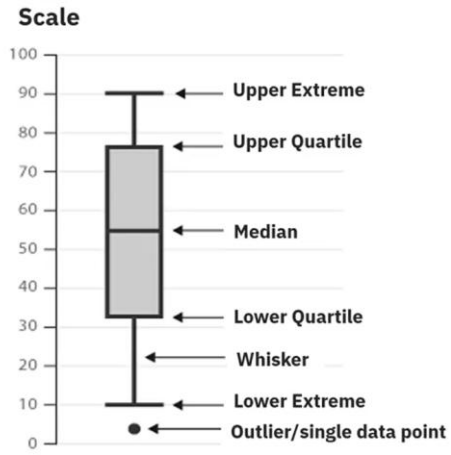
- fwd → 118
- rwd → 75
- 4wd → 8

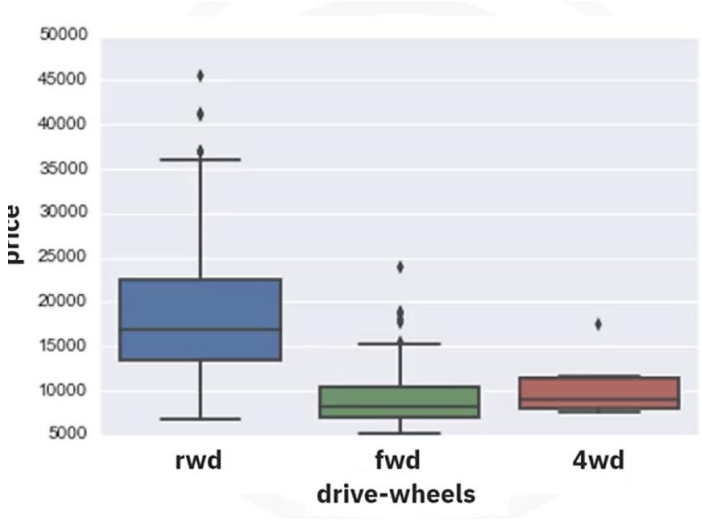
3. Box Plot

Used to:

- Compare distributions
- Detect outliers
- View spread/skewness

```
import seaborn as sns

sns.boxplot(x="drive-wheels", y="price", data=df)
```





Important Parts:

- Median → middle line
- Quartiles → box edges
- IQR → middle 50%
- Outliers → dots outside whiskers

4. Scatter Plot

Used to study relationships between continuous variables.

Example:

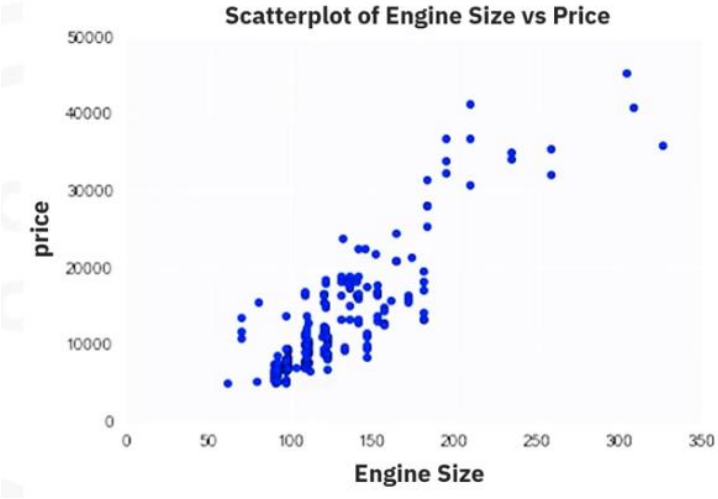
- Engine Size vs Price

```
from matplotlib import pyplot as plt

plt.scatter(df["engine-size"], df["price"])
plt.xlabel("Engine Size")
plt.ylabel("Price")
plt.title("Engine Size vs Price")
plt.show()
```

Interpretation:

- Upward trend → Positive correlation
- Downward trend → Negative correlation

- Random pattern → Weak/no correlation



3.2 GROUPBY

Used to group categorical data.

Example:
Average price by drive wheels and body style.

```
df_group = df[['drive-wheels','body-style','price']]

grouped = df_group.groupby(
    ['drive-wheels','body-style'],
    as_index=False
).mean()

print(grouped)
```

Pivot Table

Transforms grouped data into matrix form.

```
grouped_pivot = grouped.pivot(
    index='drive-wheels',
    columns='body-style'
)

print(grouped_pivot)
```

	price				
body-style	convertible	hardtop	hatchback	sedan	wagon
drive-wheels					
4wd	0.0	0.000000	7603.000000	12647.333333	9095.750000
fwd	11595.000000	8429.000000	8396.387755	9811.8000000	9997.333333
rwd	23949.600000	24202.714286	14337.777778	21711.833333	16994.222222

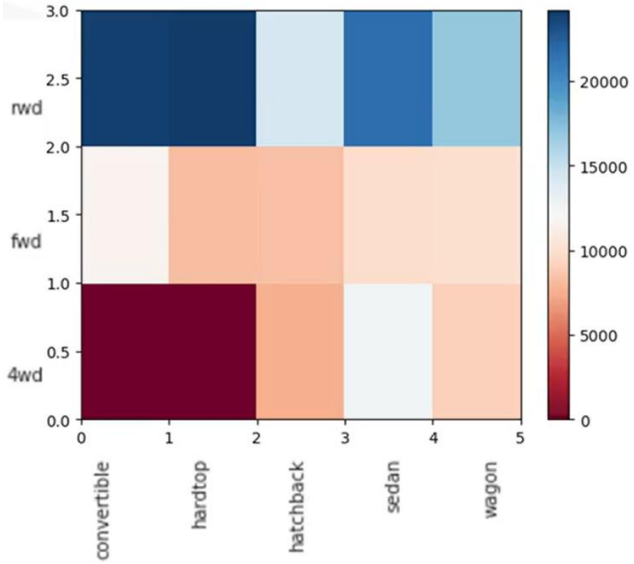
Heatmap / Pcolor Plot

Visualizes pivot tables using colors.

```
from matplotlib import pyplot as plt

plt.pcolor(grouped_pivot, cmap='RdBu')
plt.colorbar()
plt.show()
```

Interpretation:

- Darker/hotter color → higher values
- Cooler color → lower values



3.3 CORRELATION

Measures relationship between variables.

Types:

Positive Correlation

Both variables increase together.

Example:

- Engine size ↑ → Price ↑

Negative Correlation

One increases while the other decreases.

Example:

- Highway MPG ↑ → Price ↓

Weak Correlation

No clear relationship.

1. Pearson Correlation

Measures strength of linear relationship.

```
from scipy import stats

pearson_coef, p_value = stats.pearsonr(
   df['horsepower'],
   df['price']
)

print(pearson_coef)
print(p_value)
```

Pearson Coefficient Interpretation

Value	Meaning
Close to +1	Strong positive correlation
Close to -1	Strong negative correlation
Close to 0	Weak/no correlation

P-value Interpretation

P-value	Certainty
< 0.001	Strong certainty
0.001 – 0.05	Moderate certainty
0.05 – 0.1	Weak certainty
> 0.1	No certainty

Strong Correlation:

Correlation coefficient = close to +1 or -1

Pvalue < 0.001

2. Correlation Heatmap

Shows correlations among all variables.

```
import seaborn as sns

corr = df.corr()

sns.heatmap(corr)
```

Interpretation:

- Dark/high values → strong correlation
- Diagonal always = 1

3.4 IMPORTANT VISUALIZATION LIBRARIES

Matplotlib

```
from matplotlib import pyplot as plt
%matplotlib inline
```

Seaborn

```
import seaborn as sns
```

Important Plot Functions

Plot	Function
Line Plot	plt.plot()
Scatter Plot	plt.scatter()
Histogram	plt.hist()
Bar Plot	plt.bar()
Heatmap	plt.pcolor()
Regression Plot	sns.regplot()
Box Plot	sns.boxplot()
Residual Plot	sns.residplot()
KDE Plot	sns.kdeplot()
Distribution Plot	sns.distplot()

3.5 CREATING DIFFERENT TYPES OF PLOTS IN PYTHON

Data Visualization commands in Python

Visualizations play a key role in data analysis. In this reading, you'll be introduced to various forms of graphs and plots that you can create with your data in Python that help you in visualising your data for better analysis.

The two major libraries used to create plots are matplotlib and seaborn. We will learn the prominent plotting functions of both these libraries as applicable to Data Analysis.

MATPLOTLIB FUNCTIONS

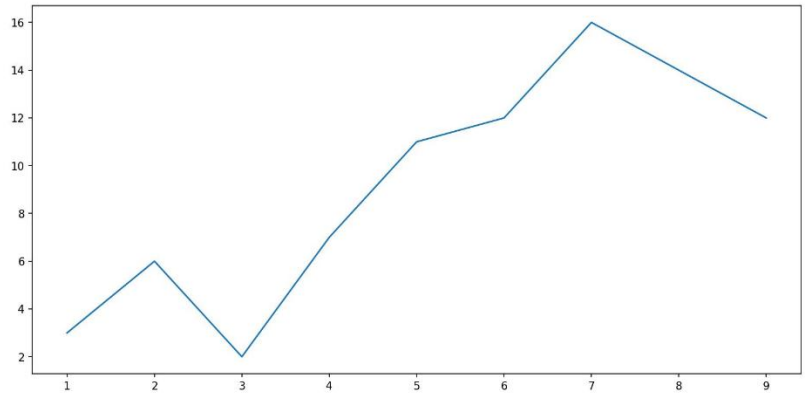
1. Standard Line Plot

The simplest and most fundamental plot is a standard line plot. The function expects two arrays as input, x and y, both of the same size. x is treated as an independent variable and y as the dependent one. The graph is plotted as shortest line segments joining the x,y point pairs ordered in terms of the variable x.

Syntax:

```
plt.plot(x,y)
```

A sample plot is shown in the image below.



2. Scatter plot

Scatter plots are graphs that present the relationship between two variables in a data set. It represents data points on a two-dimensional plane. The independent variable or attribute is plotted on the X-axis, while the dependent variable is plotted on the Y-axis.

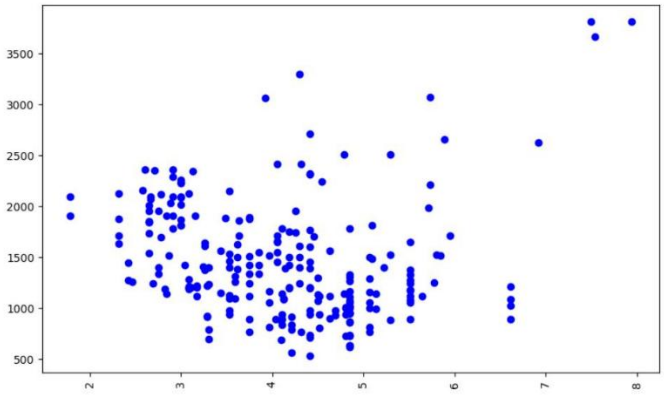
Scatter plots are used in either of the following situations:

- When we have paired numerical data
- When there are multiple values of the dependent variable for a unique value of an independent variable
- In determining the relationship between variables in some scenarios

Syntax:

```
plt.scatter(x,y)
```

Here, x contains the independent variable, and y contains the dependent variable. You have the option to change the size, color, and shape of the markers with additional attributes in the function.A sample scatter plot is shared below.



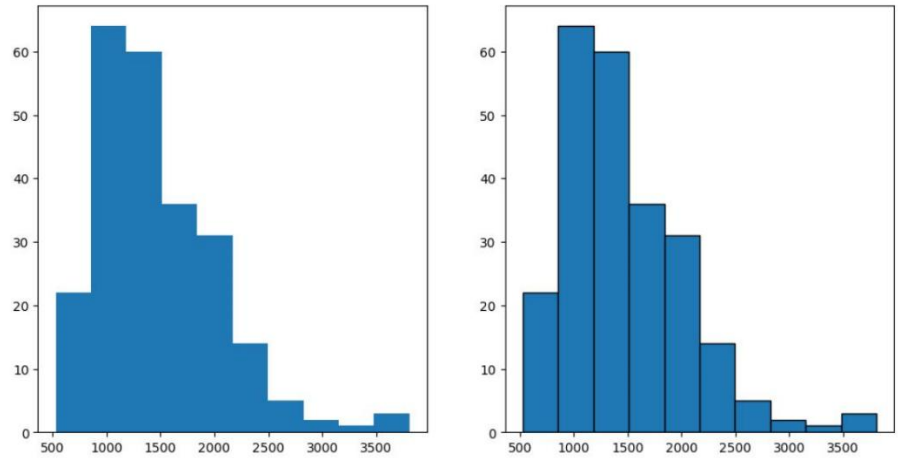
3. Histogram

A histogram is an important visual representation of data in categorical form. To view the data in a "Binned" form, we may use the histogram plot with a number of bins required or even with the data points that mark the bin edges. The x-axis represents the data bins, and the y-axis represents the number of elements in each of the bins.

Syntax:

```
plt.hist(x,bins)
```

An example of a histogram plot is shown below. Use an additional argument, edgecolor, for better clarity of plot. Consider the graph shown below. The left graph is the histogram plot for a data set, plotted without setting the edgecolor. The right one is the same graph but has the edgecolor argument set as the color black.



4. Bar plot

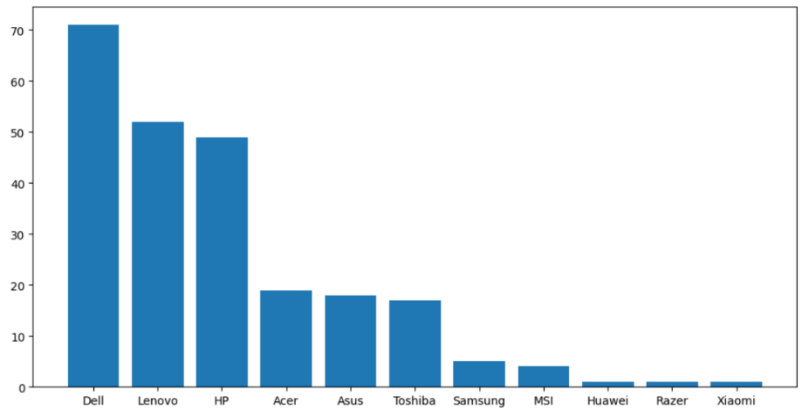
A bar plot is used for visualizing catogorical data. The y-axis represents the average value of data points belonging to a particular category, while the x-axis represents the number of elements in the different categories.

Syntax:

```
plt.bar(x,height)
```

Here, x is the categorical variable, and height is the number of values belonging to the category. You can adjust the width of each bin using an additional width argument in the function.

A sample graph is shown below.



5. Pseudo Color Plot

A pseudocolor plot displays matrix data as an array of colored cells (known as faces). This plot is created as a flat surface in the x-y plane. The surface is defined by a grid of x and y coordinates that correspond to the corners (or vertices) of the faces. Matrix C specifies the colors at the vertices. The color of each face depends on the color of one of its four surrounding vertices. Of the four vertices, the one that comes first in the x-y grid determines the color of the face.

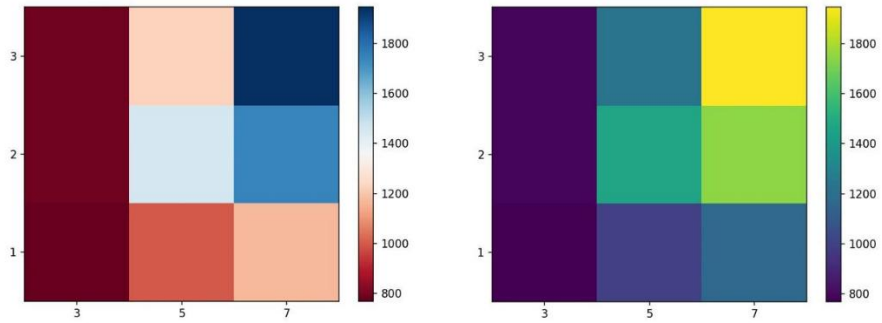
In this course, you use the pcolor plot for visualizing the contents of a pivot table that has been grouped on the basis of 2 parameters. Those parameters then represent the x and y-axis components that create the grid. The values in the pivot table are the average values of a third parameter. These values act as the code for the color the cell is going to take.

Syntax:

```
plt.pcolor(C)
```

You can define an additional cmap argument to specify the color scheme of the plot.

Two sample pcolor plots are shown below, created for same data but for different color schemes.



SEABORN FUNCTIONS

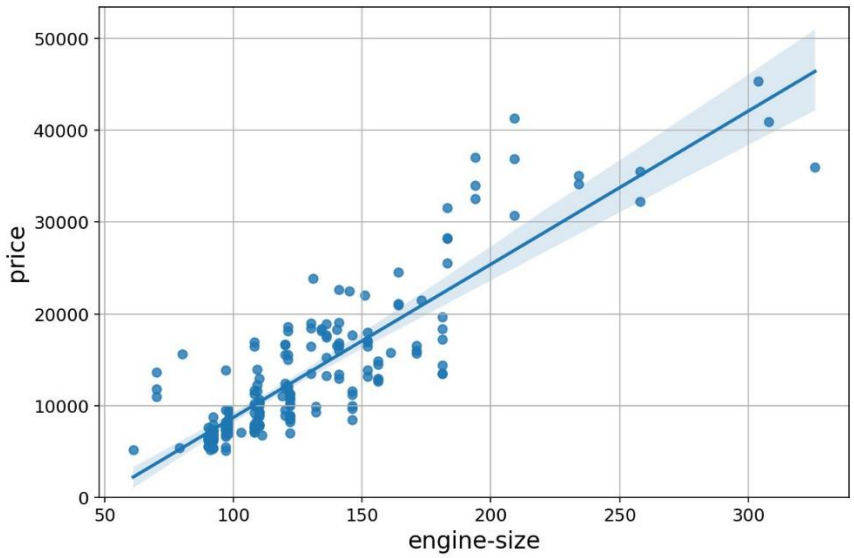
1. Regression plot

A regression plot draws a scatter plot of two variables, x and y, and then fits the regression model and plots the resulting regression line along with a 95% confidence interval for that regression. The x and y parameters can be shared as the dataframe headers to be used, and the data frame itself is passed to the function as well.

Syntax:

```
sns.regplot(x = 'header_1',y = 'header_2',data= df)
```

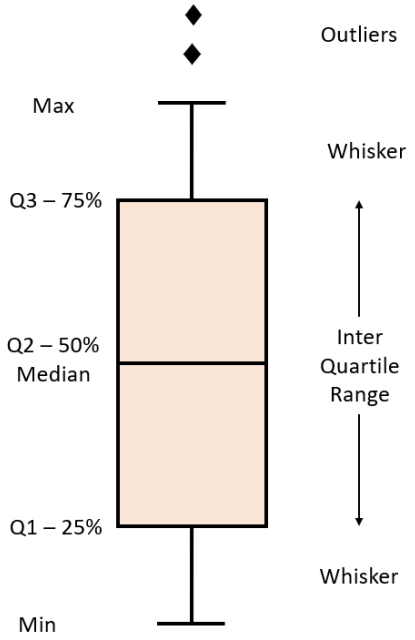
A sample regression plot is shared below.



2. Box and whisker plot

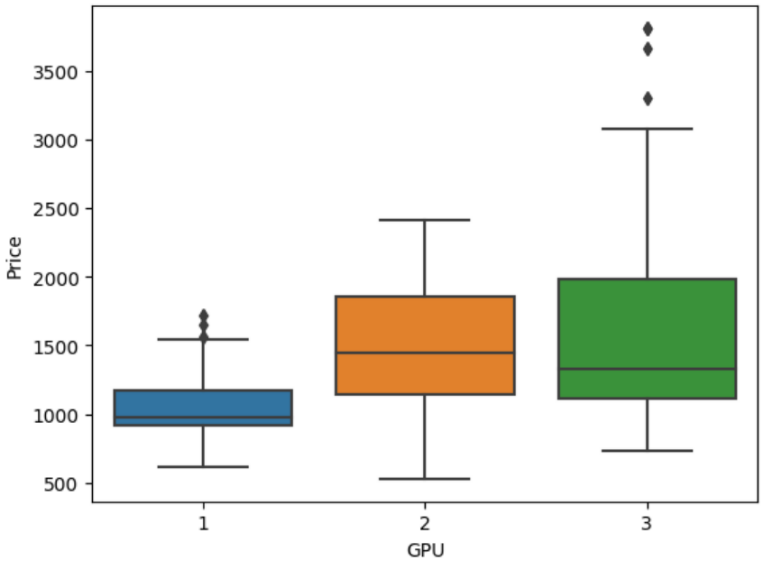
A box plot (or box-and-whisker plot) shows the distribution of quantitative data in a way that facilitates comparisons between variables or across levels of a categorical variable. The box shows the quartiles of the dataset while the whiskers extend to show the rest of the distribution, except for points that are determined to be "outliers".

Consider the Box and whisker plot interpretation figure shown below.



The plot uses whiskers to represent Minimum value to 25% quartile data and 75% quartile to Maximum value data. The range between 25% quartile and 75% quartile is considered as the Inter-Quartile Range. Outliers are generally classified as being outside 1.5 times the interquartile range.

A sample box plot is shown below



3. Residual Plot

A residual plot is used to display the quality of polynomial regression. This function will regress y on x as a polynomial regression and then draw a scatterplot of the residuals.Residuals are the differences between the observed values of the dependent variable and the predicted values obtained from the regression model. In other words, a residual is a measure of how much a regression line vertically misses a data point, meaning how far off the predictions are from the actual data points.

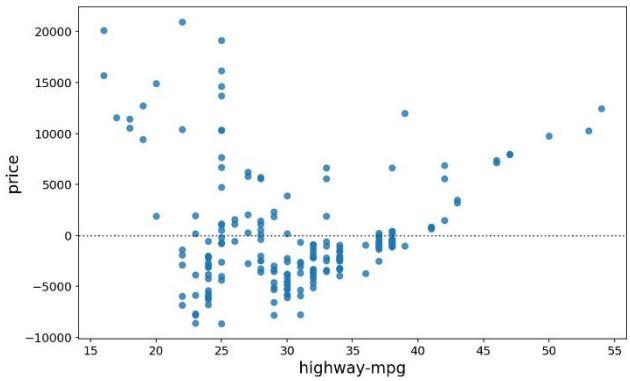
Syntax:

```
sns.residplot(data=df,x='header_1', y='header_2')
```

Alternatively:

```
sns.residplot(x=df['header_1'], y=df['header_2'])
```

A sample plot is shown below.



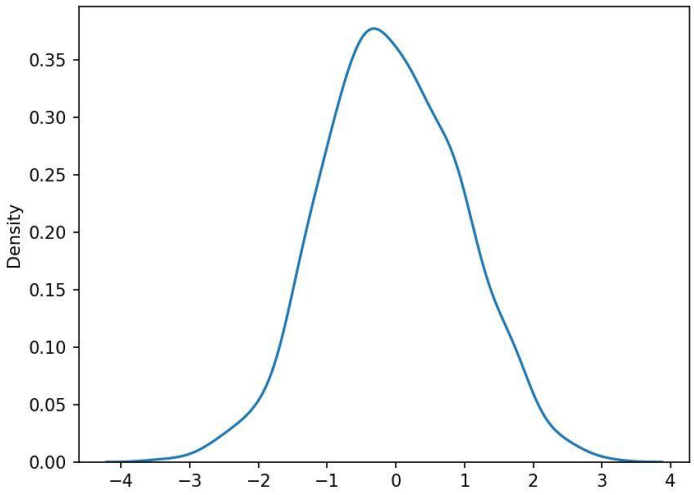
4. KDE plot

A Kernel Density Estimate (KDE) plot is a graph that creates a probability distribution curve for the data based upon its likelihood of occurrence on a specific value. This is created for a single vector of information. It is used in the course in order to compare the likely curves of the actual data with that of the predicted data.

Syntax:

```
sns.kdeplot(X)
```

A sample graph made for a random set of values is shown below.



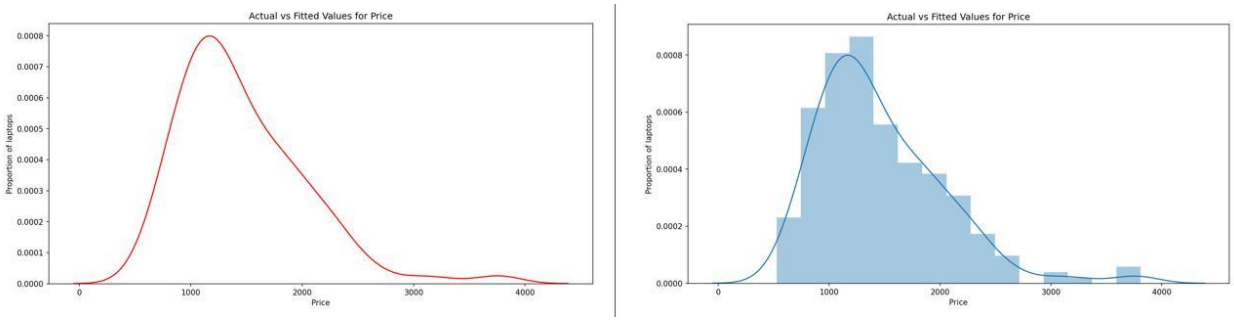
5. Distribution Plot

This plot has the capacity to combine the histogram and the KDE plots. This plot creates the distribution curve using the bins of the histogram as a reference for estimation. You can optionally keep or discard the histogram from being displayed. In the context of the course, this plot can be used interchangeably with the KDE plot.

Syntax:

```
sns.distplot(X,hist=False)
```

Here, keeping the argument hist as True would plot the histogram along with the distribution plot. Both variations are shown in the image below.



3.6 CHI-SQUARE TEST

Used for categorical variables.

Checks whether two categorical variables are associated.

1. Hypotheses

Null Hypothesis (H_0)

No association exists.

Alternative Hypothesis (H_1)

Association exists.

2. Python Example

```
import pandas as pd
from scipy.stats import chi2_contingency
data = [[20,30], [25,25]]
df = pd.DataFrame(
    data,
    columns=["Like","Dislike"],
    index=["Male","Female"])
chi2, p, dof, expected = chi2_contingency(df)

print("Chi-square:", chi2)
print("P-value:", p)
```

3. Chi-Square Interpretation

Condition	Conclusion
$p < 0.05$	Significant association
$p > 0.05$	No significant association

FINAL KEY CONCEPTS

Concept	Purpose
EDA	Understand data
Scatter Plot	Relationship between variables
Box Plot	Distribution + Outliers
GroupBy	Analyze categories
Correlation	Variable relationship
Pearson	Strength of correlation
Heatmap	Visual correlation matrix
Chi-Square	Association between categorical variables

LAB: EXPLORATORY DATA ANALYSIS - USED CAR PRICING

LAB: EXPLORATORY DATA ANALYSIS - LAPTOP PRICING

MODULE 4: MODEL DEVELOPMENT

1. Introduction to Model Development

What is Model Development?

Model development is the process of creating mathematical models that predict a target value using input features.

Example

Predicting the **price of a car** using:

- Highway MPG
- Engine size
- Horsepower
- Age

- Brand

Terminology

Term	Meaning
Feature / Independent Variable (X)	Input variable used for prediction
Target / Dependent Variable (Y)	Output variable being predicted
Model / Estimator	Mathematical relationship between X and Y

2. Why More Relevant Data Matters

A model becomes more accurate when:

- More useful features are included
- Data represents real-world situations

Example

If car color affects price:

- Pink cars sell cheaper
- Red cars sell higher

If the model does NOT include color:

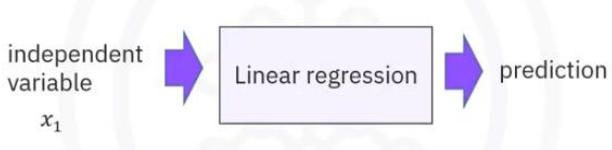
- Both cars get same predicted price
- Prediction becomes inaccurate

4.1 REGRESSION MODELS

1. TYPES REGRESSION MODELS

A. Simple Linear Regression (SLR)

Uses **one independent variable** to predict the target.



Formula

$$\hat{y} = b_0 + b_1 x$$

Where:

- $\hat{y}$= predicted value
- b_0= intercept
- b_1= slope
- x= feature

Example

Predict car price using:

- Highway MPG only

B. Multiple Linear Regression (MLR)

Uses **multiple features** to predict the target.

Formula

$$\hat{y} = b_0 + b_1 x_1 + b_2 x_2 + b_3 x_3 + \ldots$$

Example

Predict price using:

- Horsepower
- Engine size
- Highway MPG
- Curb weight

2. Noise in Regression

Real-world data contains randomness called **noise**.

Causes

- Human behavior
- Missing variables
- Market fluctuations
- Measurement errors

Thus:

- Predictions are not perfectly accurate
- Errors are expected

3. Linear Regression in Python

Import Library

```
from sklearn.linear_model import LinearRegression
```

Create Model

```
lm = LinearRegression()
```

Define Variables

```
X = df[['highway-mpg']]
Y = df['price']
```

Train Model

```
lm.fit(X, Y)
```

Predict

```
Yhat = lm.predict(X)
```

4. Model Parameters

Intercept

```
lm.intercept_
```

Coefficients (Slope)

```
lm.coef_
```

Example Equation

$$price = 38423.31 - 821.73 \times highway\text{-}mpg$$

Meaning:

- Every increase of 1 MPG decreases car price by approximately $821.

5. Multiple Linear Regression in Python

Define Multiple Features

```
Z = df[['horsepower', 'curb-weight',
       'engine-size', 'highway-mpg']]
```

Train Model

```
lm.fit(Z, df['price'])
```

Predict

```
Yhat = lm.predict(Z)
```

4.2 MODEL EVALUATION USING VISUALIZATION

Visualization helps determine:

- Whether the model fits data properly
- Whether assumptions are correct

1. Regression Plot

Purpose

Shows:

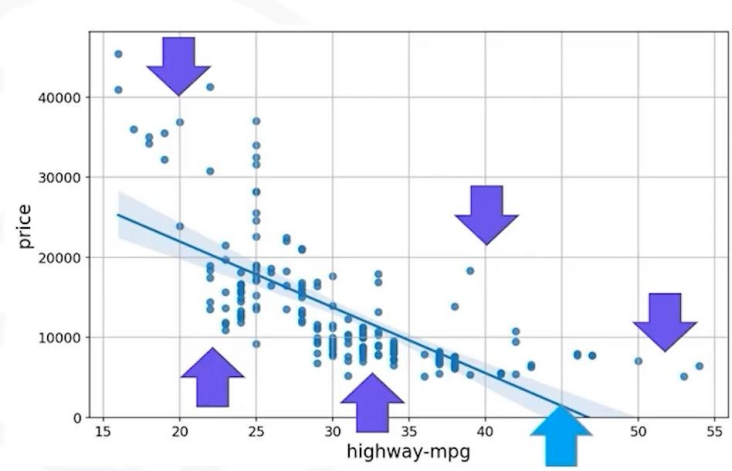
- Relationship strength
- Positive/negative correlation
- Linearity

Code

```
import seaborn as sns

sns.regplot(x='highway-mpg',
        y='price',
        data=df)

plt.ylim(0, )
```

Interpretation

Observation	Meaning
Downward slope	Negative relationship
Upward slope	Positive relationship
Tight clustering	Strong correlation
Scattered points	Weak correlation



2. Residual Plot

Residual

$$Residual = Actual - Predicted$$

Purpose

Checks:

- Model correctness
- Linearity assumption

Code

```
sns.residplot(x=df['highway-mpg'],
              y=df['price'])
```

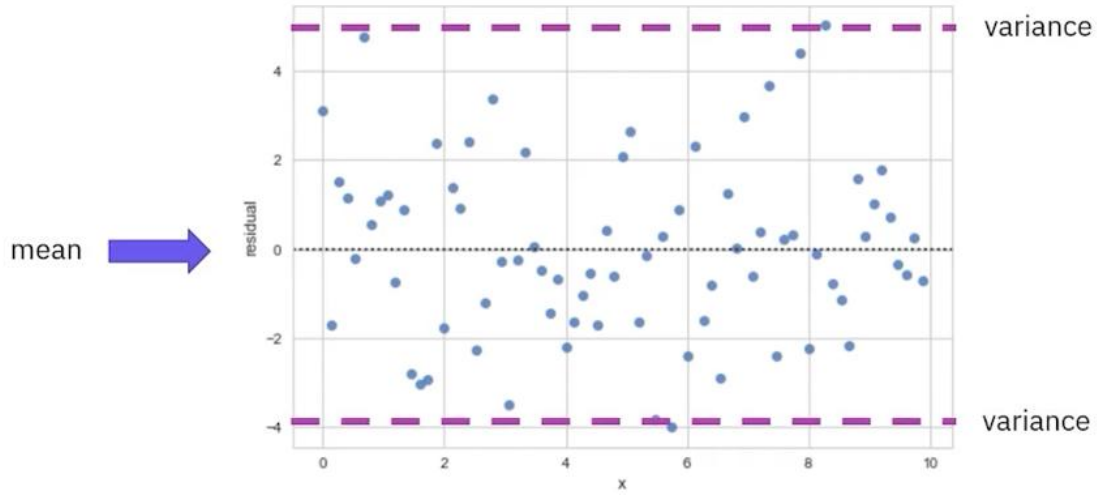
Good Residual Plot

Characteristics:

- Residuals randomly scattered
- Mean around zero
- Constant variance
- No clear pattern

Meaning

Linear regression is appropriate.



Bad Residual Plot

Curved Pattern

Indicates:

- Nonlinear relationship

Funnel Shape

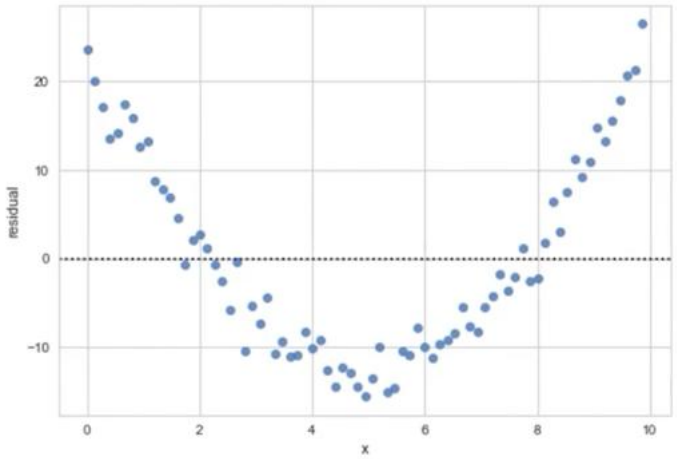
Indicates:

- Changing variance (heteroscedasticity)

Clusters

Indicates:

- Missing variables



3. Distribution Plot

Purpose

Compares:

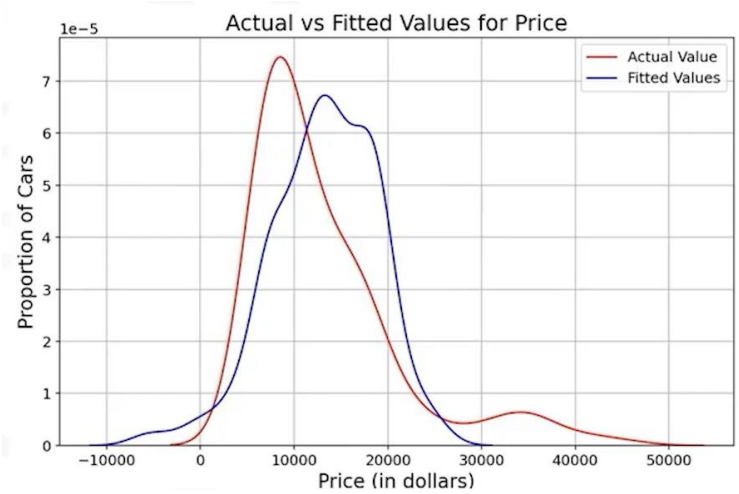
- Actual values
- Predicted values

Useful for:

- Multiple Linear Regression

Code

```
ax1 = sns.kdeplot(df['price'],
                  color='r',
                  label='Actual Values')

sns.kdeplot(Yhat,
            color='b',
            label='Fitted Values',
            ax=ax1)
```



4. KDE Plot (Kernel Density Estimation)

Why KDE Instead of Histogram?

KDE	Histogram
Smooth curve	Uses bars
Better distribution visualization	Sensitive to bin size
Modern replacement	Less flexible

KDE Plot Interpretation

Good Model

- Curves overlap significantly

Poor Model

- Curves differ widely
- Peaks shift
- Predicted spread narrower

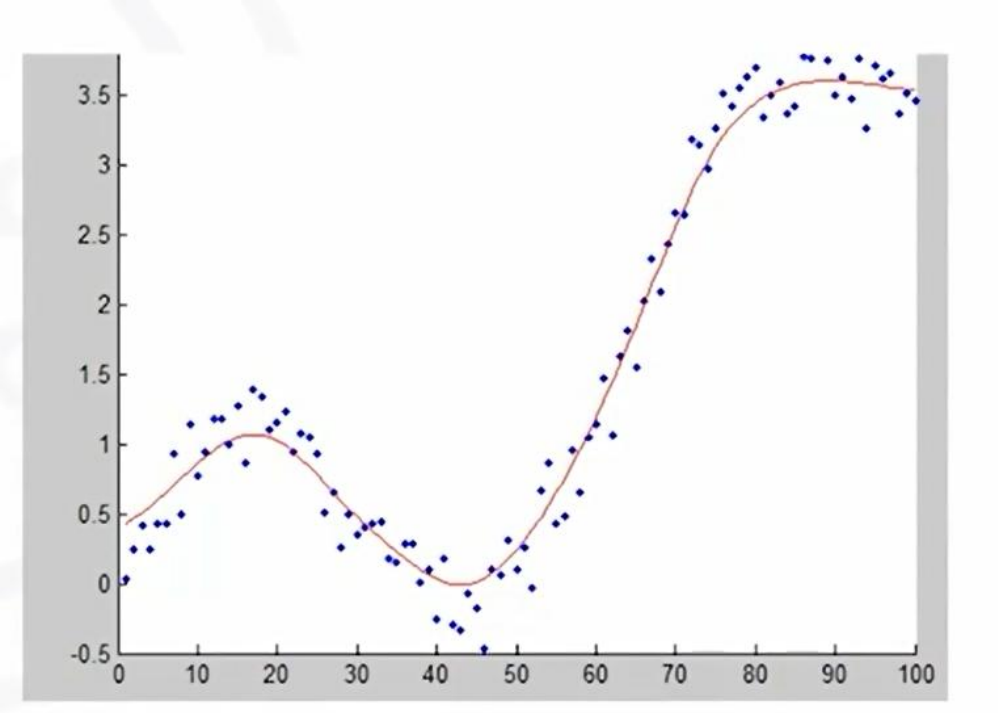
4.3 POLYNOMIAL REGRESSION

1. Why Needed?

Linear models fail for curved relationships.

Polynomial regression captures:

- Curvature
- Complex patterns



2. Polynomial Forms

Quadratic (2nd Order)

$$y = b_0 + b_1x + b_2x^2$$

Cubic (3rd Order)

$$y = b_0 + b_1x + b_2x^2 + b_3x^3$$

3. Polynomial Regression in Python

Create Polynomial

```
f = np.polyfit(x, y, 3)
```

Create Model

```
p = np.poly1d(f)
```

Predict

```
Yhat = p(x)
```

4. Multivariate Polynomial Regression

Import

```
from sklearn.preprocessing import PolynomialFeatures
```

Create Features

```
pr = PolynomialFeatures(degree=2)
```

Transform Data

```
Z_pr = pr.fit_transform(Z)
```

5. Feature Scaling / Normalization

Why Normalize?

Features may have:

- Different ranges
- Different units

Example:

- Horsepower: 300
- MPG: 20

Large-scale features dominate learning.

StandardScaler

Import

```
from sklearn.preprocessing import StandardScaler
```

Fit & Transform

```
SCALE = StandardScaler()
SCALE.fit(X)
X_scale = SCALE.transform(X)
```

4.4 PIPELINES

1. Purpose

Automates:

1. Scaling
2. Polynomial transformation
3. Model fitting
4. Prediction

2. Pipeline in Python

```
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.preprocessing import PolynomialFeatures
from sklearn.linear_model import LinearRegression
```

Create Pipeline

```
Input = [
   ('scale', StandardScaler()),
   ('polynomial', PolynomialFeatures()),
   ('model', LinearRegression())
]
pipe = Pipeline(Input)
```

Train

```
pipe.fit(Z, y)
```

Predict

```
ypipe = pipe.predict(Z)
```

4.5 KERNEL DENSITY ESTIMATION (KDE) PLOTS FOR MODEL EVALUATION

1. Introduction

- Kernel Density Estimation (KDE) plots are a valuable tool for visualizing data distributions by estimating their probability density function (PDF).
- These plots are particularly useful in regression analysis for comparing actual and predicted values.
- With the deprecation of Seaborn distplot, KDE plots serve as a modern and effective method for assessing model performance.

2. Why Use KDE Plots?

KDE plots are beneficial in model evaluation for the following reasons:

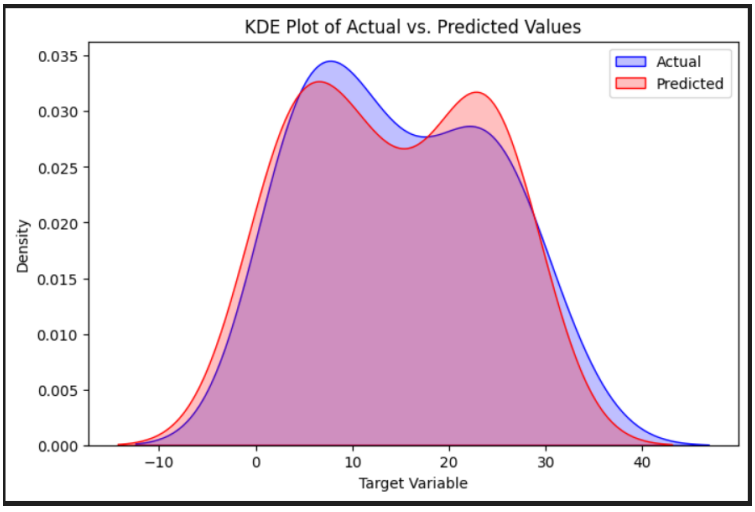
- They provide a smooth approximation of the data distribution.
- They help compare the true vs. predicted distributions effectively.
- Unlike histograms, KDE plots are not sensitive to bin sizes.
- They can highlight deviations between observed and predicted values.

3. Implementing KDE Plots in Python

```
import seaborn as sns

import matplotlib.pyplot as plt

from sklearn.model_selection import train_test_split

from sklearn.linear_model import LinearRegression

from sklearn.metrics import mean_absolute_error

# Generating Sample Data

npy.random.seed(42)

x = npy.random.rand(100) * 10

y = 3 * x + npy.random.normal(0, 3, 100)  # Linear relation with noise

data = pds.DataFrame({'X': x, 'Y': y})
```

```
# Splitting Data
X_train, X_test, y_train, y_test = train_test_split(data[['X']], data['Y'], test_size=0.2, random_state=42)
# Training a Model
model = LinearRegression()
model.fit(X_train, y_train)
y_pred = model.predict(X_test)
# Plotting KDE for Observed vs. Predicted Values
plt.figure(figsize=(8, 5))
sns.kdeplot(y_test, label='Actual', fill=True, color='blue')
sns.kdeplot(y_pred, label='Predicted', fill=True, color='red')
plt.xlabel('Target Variable')
plt.ylabel('Density')
plt.title('KDE Plot of Actual vs. Predicted Values')
plt.legend()
plt.show()
```

The resulted Kernel Density Estimation (KDE) plot compares the distribution of **actual values (blue)** and **predicted values (red)** from the linear regression model.



4. Interpretation of the KDE Plot

- **Overlap Between Distributions:** The two curves have a significant overlap, indicating that the model has captured the general distribution of the actual target values reasonably well. However, the predicted values slightly deviate from the actual values in some regions.
- **Peak Differences (Mode Shifts):** The **blue (actual)** curve peaks slightly higher than the red curve, meaning that the actual values are more concentrated around certain values. The **red (predicted**) curve has a second peak, suggesting that the model may be slightly misestimating certain ranges of the target variable.
- **Spread of the Distributions:** The **actual values (blue)** seem to have a wider spread, indicating more variation in real-world values. The **predicted values (red)** appear to be narrower, which suggests the model might be slightly underestimating variance (a sign of over-smoothing or bias).
- **Tails of the Distributions:** The tails of the predicted values closely follow the actual values, meaning the model does not generate extreme outliers beyond what was observed in the data. If there was a significant mismatch in the tails, it could indicate that the model struggles with extreme cases.

4.6 MEAN SQUARED ERROR (MSE)

1. Definition

Measures average squared difference between:

- Actual values
- Predicted values

2. Formula

$$MSE = \frac{1}{n}\sum(y - \hat{y})^2$$

3. Interpretation

MSE Value	Meaning
Low	Better model
High	Poor model

4. Python Code

```
from sklearn.metrics import mean_squared_error
mse = mean_squared_error(y, Yhat)
```

4.7 R-SQUARED (COEFFICIENT OF DETERMINATION)

1. Purpose

Measures how much variation in Y is explained by X.

2. Formula

$$R^2 = 1 - \frac{MSE_{model}}{MSE_{mean}}$$

3. Interpretation

R^2 Value	Meaning
1	Perfect fit
Near 1	Strong model
Near 0	Weak model
Negative	Possible overfitting

4. Python Code

```
R2 = lm.score(X, Y)
```

5. Interpreting R^2

R^2	Interpretation
0.998	Excellent
0.92	Strong
0.80	Moderate
0.60	Weak but visible relationship

4.8 MODEL EVALUATION STRATEGY

Always use:

1. Visualization
2. Numerical metrics
3. Compare multiple models

1. Comparing Models

Important Concept

Lower MSE does NOT always mean better model.

Why?

- Adding more variables reduces MSE
- Complex models overfit

2. Overfitting

Definition

Model memorizes training data instead of learning patterns.

Symptoms

- Very high R^2 on training data
- Poor real-world predictions

- Negative R^2 on test data

4.9 PREDICTION AND DECISION MAKING

Example

Predict car price at:

```
lm.predict([[30]])
```

Output:

```
13771.30
```

1. Sensibility Check

Always verify predictions:

- Not negative
- Not unrealistically high
- Fits domain knowledge

2. Generating Sequences with NumPy

```
import numpy as np

new_input = np.arange(1, 100, 1)
```

Parameters

Parameter	Meaning
Start	1
Stop	100
Step	1

KEY DIFFERENCES BETWEEN MODELS

Model	Features	Captures Curves?
Simple Linear Regression	1	No
Multiple Linear Regression	Multiple	No
Polynomial Regression	1 or more	Yes

IMPORTANT VISUALIZATION TECHNIQUES

Plot	Purpose
Regression Plot	Relationship strength
Residual Plot	Error analysis
Distribution/KDE Plot	Compare actual vs predicted

COMPLETE WORKFLOW

Step-by-Step

1. Import dataset
2. Select features
3. Train model
4. Predict values
5. Visualize results
6. Calculate MSE
7. Calculate R^2
8. Improve model if necessary

LAB: MODEL DEVELOPMENT - USED CAR PRICING

LAB: MODEL DEVELOPMENT - LAPTOP PRICING

MODULE 5: MODEL EVALUATION AND REFINEMENT

Introduction to Model Evaluation

What is Model Evaluation?

Model evaluation determines **how well a machine learning model performs on unseen data**.

In Module 4:

- We focused on **in-sample evaluation**
- Model was evaluated using the same data used for training

Problem:

- A model may perform well on training data
- But fail on new real-world data

This problem is solved using:

- Train/Test Split
- Cross Validation
- Regularization
- Hyperparameter Tuning

5.1 TRAINING DATA VS TESTING DATA

1. Why Split Data?

If the model is tested using the same data it learned from:

- Performance appears artificially high
- The model may memorize patterns instead of learning general relationships

Therefore:

- Training Set → Used to train the model
- Testing Set → Used to evaluate model performance

2. Train-Test Split

Concept

Dataset is divided into:

- Training data
- Testing data

Common splits:

- 70% Training / 30% Testing
- 80% Training / 20% Testing
- 90% Training / 10% Testing

3. Workflow

Step 1: Train Model

Use training data to:

- Discover relationships
- Learn coefficients

Step 2: Test Model

Use unseen test data to:

- Measure prediction quality
- Estimate real-world performance

4. train_test_split() in Scikit-learn

Import

```
from sklearn.model_selection import train_test_split
```

Example

```
y_data = df['price']
x_data = df.drop('price', axis=1)
x_train, x_test, y_train, y_test = train_test_split(
    x_data,
    y_data,
    test_size=0.3,
    random_state=1
)
```

Explanation

Parameter	Meaning
x_data	Input features
y_data	Target variable
test_size=0.3	30% data for testing
random_state	Ensures reproducibility

5.2 GENERALIZATION ERROR

1. Definition

Generalization error measures:

How well the model predicts unseen data

Important Point

Training accuracy ≠ Real-world accuracy

A model may:

- Fit training data perfectly
- Fail on unseen data

Testing data approximates real-world performance.

2. Tradeoff in Train/Test Split

More Training Data

Advantages

- Better learning
- Better model fitting

Disadvantages

- Smaller testing set
- Less reliable evaluation

More Testing Data

Advantages

- Better evaluation precision

Disadvantages

- Less training data
- Poorer model learning

3. Cross Validation

Why Cross Validation?

Single train-test split may:

- Depend heavily on random data split
- Produce unstable results

Cross-validation provides:

- More reliable evaluation
- Better estimate of model performance

4. K-Fold Cross Validation

Concept

Dataset is divided into **K folds**.

Example:

- K = 4

Process:

1. Use 3 folds for training
2. Use 1 fold for testing
3. Repeat until every fold is used as test data

Final result:

- Average of all evaluation scores

Example

Fold	Training	Testing
1	2,3,4	1
2	1,3,4	2
3	1,2,4	3
4	1,2,3	4

5. cross_val_score()

Purpose

Performs cross-validation and returns evaluation scores.

Import

```
from sklearn.model_selection import cross_val_score
from sklearn.linear_model import LinearRegression
```

Example

```
lr = LinearRegression()
scores = cross_val_score(
    lr,
    x_data,
    y_data,
    cv=4
)
```

Output

```
array([0.74, 0.69, 0.71, 0.73])
```

Each value:

- R^2 score for one fold

Average Score

```
scores.mean()
```

Standard Deviation

```
scores.std()
```

Lower standard deviation:

- More stable model

6. cross_val_predict()

Purpose

Returns predictions generated during cross-validation.

Example

```
from sklearn.model_selection import cross_val_predict
yhat = cross_val_predict(
    lr,
    x_data,
    y_data,
    cv=4
)
```

Difference Between Functions

Function	Output
cross_val_score()	Evaluation scores
cross_val_predict()	Predicted values

5.3 OVERFITTING, UNDERFITTING AND MODEL SELECTION

1. Underfitting

Definition

Underfitting occurs when:

Model is too simple to capture data patterns

Characteristics

- High training error
- High testing error
- Poor predictions

Example

Using linear regression for highly curved data.

Causes

- Model too simple
- Low polynomial degree
- Important features missing

2. Overfitting

Definition

Overfitting occurs when:

Model memorizes training data and noise

Characteristics

- Very low training error
- High testing error
- Poor generalization

Example

16th-order polynomial:

- Fits every training point
- Oscillates heavily
- Performs poorly on new data

3. Model Selection

Goal

Choose model complexity that:

- Minimizes test error
- Balances bias and variance

Polynomial Regression Review

Polynomial Equation

Quadratic

$y = b0 + b1x + b2x^2$

Cubic

$y = b0 + b1x + b2x^2 + b3x^3$

Higher degree:

- More flexibility
- Higher overfitting risk

Polynomial Degree Selection

Key Observation

Training Error

Always decreases as degree increases.

Test Error

- Decreases initially
- Reaches minimum
- Then increases due to overfitting

Best Polynomial Degree

Chosen at:

Minimum test error

Irreducible Error

Definition

Some prediction error is unavoidable because of:

- Noise
- Randomness
- Missing information

Even perfect models cannot eliminate this.

Evaluating Polynomial Models Using R^2

Example Workflow

```
Rsqu_test = []
orders = [1,2,3,4,5
for n in orders
   pr = PolynomialFeatures(degree=n)
   x_train_pr = pr.fit_transform(x_train
   x_test_pr = pr.fit_transform(x_test)
   lr.fit(x_train_pr, y_train)
   Rsqu_test.append(
      lr.score(x_test_pr, y_test)
   )
```

5.4 RIDGE REGRESSION

1. What is Ridge Regression?

Ridge regression:

Prevents overfitting by shrinking coefficients

Used especially when:

- Polynomial features exist
- Multiple correlated features exist

2. Problem Without Ridge Regression

High-order polynomial models may:

- Produce huge coefficients
- Become unstable
- Overfit noise

3. Ridge Regression Formula Idea

Adds penalty term:

$$\text{Loss} = \text{MSE} + \alpha(\text{coefficient}^2)$$

Where:

- α (alpha) controls regularization strength

4. Effect of Alpha (α)

Alpha Value	Effect
Very small	Little regularization
Moderate	Controls overfitting
Very large	Underfitting

5. Ridge Regression in Python

Import

```
from sklearn.linear_model import Ridge
```

Example

```
RidgeModel = Ridge(alpha=1)
```

Training

```
RidgeModel.fit(x_train, y_train)
```

Prediction

```
yhat = RidgeModel.predict(x_test)
```

6. Ridge Regression with Polynomial Features

Example

```
from sklearn.preprocessing import PolynomialFeatures
pr = PolynomialFeatures(degree=2)
x_train_pr = pr.fit_transform(x_train)
x_test_pr = pr.fit_transform(x_test)
RidgeModel = Ridge(alpha=1)
RidgeModel.fit(x_train_pr, y_train)
yhat = RidgeModel.predict(x_test_pr)
```

7. Validation Data

Purpose

Validation data is used for:

- Hyperparameter tuning
- Model selection

NOT final testing.

8. Hyperparameters

Definition

Parameters set BEFORE training.

Examples:

- Polynomial degree
- Alpha in Ridge Regression
- Number of folds in CV

5.5 GRID SEARCH

1. What is Grid Search?

Grid Search automatically tests:

- Multiple hyperparameter combinations
- Using cross-validation

Finds:

Best hyperparameter values

2. GridSearchCV

Import

```
from sklearn.model_selection import GridSearchCV
```

3. Grid Search Example

```
from sklearn.linear_model import Ridge
parameters = [
   {'alpha':[0.001,0.1,1,10,100]}
]
RR = Ridge()
Grid1 = GridSearchCV(
   RR,
   parameters,
   cv=4)
Grid1.fit(x_data, y_data)
```

---

4. Best Model

Best Estimator

```
BestRR = Grid1.best_estimator_
```

Best Score

```
Grid1.best_score_
```

Best Parameters

```
Grid1.best_params_
```

5. Multiple Hyperparameters

Example

```
parameters = [
{
   'alpha':[0.001,0.1,1,10],
   'normalize':[True, False]
}
]
```

Grid Search tries every combination.

6. Model Evaluation Metrics

Important Metrics

Mean Squared Error (MSE)

Measures prediction error.

Lower is better.

R^2 Score

Measures explained variance.

Closer to 1 is better.

7. Comparing Training and Test Scores

Situation	Training Score	Test Score
Good Fit	High	High
Underfitting	Low	Low
Overfitting	Very High	Low

8. Bias-Variance Tradeoff

Underfitting

- High Bias
- Low Variance

Overfitting

- Low Bias
- High Variance

Goal:

- Balance both

5.6 COMPLETE MODEL REFINEMENT WORKFLOW

Step-by-Step

1. Split Data

```
train_test_split()
```

2. Train Model

```
fit()
```

3. Evaluate Model

- MSE
- R^2

- Residual plots

4. Apply Cross Validation

cross_val_score()

5. Try Polynomial Features

PolynomialFeatures()

6. Prevent Overfitting

Ridge()

7. Tune Hyperparameters

GridSearchCV()

FINAL KEY TAKEAWAYS

- Training accuracy alone is misleading
- Always use test data
- Cross-validation gives reliable evaluation
- Underfitting = model too simple
- Overfitting = model too complex
- Ridge regression reduces overfitting
- Grid Search finds best hyperparameters
- Best model balances:
    - Accuracy
    - Simplicity
    - Generalization ability

LAB: MODEL EVALUATION AND REFINEMENT - USED CARS PRICING

LAB: MODEL EVALUATION AND REFINEMENT - LAPTOP PRICING

MODULE 6: FINAL ASSIGNMENT

1. Introduction to Module 6

Module 6 combines everything learned in previous modules into **real-world end-to-end projects**.

You apply:

- Data Cleaning
- Exploratory Data Analysis (EDA)
- Linear Regression
- Polynomial Regression
- Ridge Regression
- Model Evaluation
- Model Refinement
- Visualization
- Prediction & Decision Making

This module contains:

1. **Practice Project**
    - Insurance Cost Prediction
2. **Final Project**
    - House Price Prediction

2. Practice Project — Insurance Cost Dataset

Objective

You work with an insurance dataset to predict medical insurance charges.

Main tasks:

- Load data
- Clean missing values
- Perform EDA

- Build regression models
- Improve model using Ridge Regression

3. Insurance Dataset Features

Feature	Description	Type
Age	Age of insured person	Integer
Gender	Female=1, Male=2	Numerical
BMI	Body Mass Index	Float
No_of_Children	Number of children	Integer
Smoker	Smoker=1, Non-smoker=2	Numerical
Region	Geographic region	Numerical
Charges	Insurance charges (Target)	Float

4. End-to-End Workflow

Complete Data Analysis Pipeline

1. Load Data
2. Clean Data
3. Explore Data
4. Select Features
5. Train Model
6. Predict
7. Evaluate
8. Refine Model
9. Tune Hyperparameters
10. Finalize Best Model

KEY TAKEAWAYS

- Train-test split estimates real-world performance
- Cross-validation improves reliability
- Polynomial regression models nonlinear relationships
- Ridge regression reduces overfitting
- Grid Search finds optimal hyperparameters
- R^2 and MSE are core evaluation metrics
- Visualization is critical for understanding models

PRACTICE PROJECT: INSURANCE COST ANALYSIS

FINAL PROJECT: HOUSE SALES IN KING COUNTY, USA
